

TD 10 : Tri de tableau et recherche dichotomique

Exercice 1 (tri par remplacement)

Dans cet exercice, nous disposons d'un type **t_tablo**, tableau permettant de représenter 100 entiers :

constante entier N := 100;

type t_tablo = **tableau**[N] **de entier**;

Remarque : on supposera que le tableau à trier est initialement complété.

L'algorithme que nous vous proposons de concevoir est celui du tri par remplacement. La procédure de tri aura deux paramètres, le tableau **tab** à trier et le tableau **tabRes** résultat.

Le principe est le suivant : il consiste à extraire du tableau en entrée les minimums successifs et de les ranger dans le tableau résultat. Chaque minimum extrait est remplacé par le plus grand élément du tableau. On peut distinguer les étapes suivantes :

- Étape 1 : recherche du plus grand élément du tableau en entrée.
- Étape 2 : recherche des n minimums successifs. Chaque minimum est rajouté dans le tableau résultat (en commençant par les cases d'indices les plus faibles).

1. **Question** : Appliquer "manuellement" le tri précédent sur l'exemple suivant ($N = 5$). Est-il croissant ou décroissant ?

8	0	9	3	5
---	---	---	---	---

2. **Question** : Écrire en langage algorithmique la procédure de tri.

Pour cela, vous développerez plusieurs fonctions ou procédures :

fonction maxTab() // qui retourne la valeur maximale du tableau

fonction indMin() // qui retourne l'indice de la valeur minimale du tableau

procédure copie(**entF** tabIn : typeTab, **sortF** tabOut : typeTab) // copie de tabIn dans tabOut

procédure triRempDec() // qui permet de trier un tableau par ordre décroissant

(bonus) **procédure** triRempCroi() // qui permet de trier un tableau par ordre croissant

Exercice 2 (tri par comptage)

<https://www.youtube.com/watch?v=Xz4lZd07VH0>

L'algorithme que nous vous proposons de concevoir est celui du tri par comptage. Ce tri est très efficace mais également très restrictif : uniquement des valeurs entières strictement positives.

Le principe consiste à construire un tableau intermédiaire **tabTemp** dont la dimension correspond à la valeur maximale du tableau à trier.

Dans un deuxième temps, il faudra parcourir le tableau à trier et incrémenter la case du tableau intermédiaire dont l'indice correspondant à la valeur du tableau initial.

La procédure de tri aura trois paramètres, le tableau **tab** à trier, le tableau **tabRes** résultat.

On peut distinguer les étapes suivantes :

- Étape 1 : les valeurs du tableau **tabTemp** sont initialisées à 0
- Étape 2 : Le tableau **tab** est parcouru et chaque valeur **tabTemp[tab[i]]** est incrémentée.
- Étape 3 : on parcourt le tableau **tabTemp** pour construire le tableau **tabRes** résultat.

1. **Question** Appliquer "manuellement" le tri précédent sur l'exemple suivant :

12	10	12	6	14
----	----	----	---	----

2. **Question** Écrire en langage algorithmique la procédure de tri d'un tableau de 10 entiers, chaque valeur pourra être comprise entre 0 et 99 (bornes incluses). L'entête de la procédure sera le suivant :

procedure triComptage(**entF** tab : **tableau de entier**, **sortF** tabRes : **tableau de entier**);

la variable **taille** correspond à la taille du tableau à trier.

Vous définirez deux constantes :

- MAX_ELT qui correspond au nombre d'éléments du tableau à trier.
- VAL_MAX qui correspond à la valeur maximale des données à trier.

Vous définirez également la procédure init() qui permettra d'initialiser l'ensemble des valeurs d'un tableau de taille VAL_MAX à 0. L'entête de la procédure sera le suivant :

procédure init(**sortF** tab : **tableau**);

Vous supposerez que la procédure gene() existe. Elle permet de générer un tableau de nombres aléatoires entiers strictement positifs et inférieurs à VAL_MAX. L'entête de la procédure est le suivant :

procédure gene(**sortF** tab : **tableau**);

3. **Question** : Écrire un programme permettant de trier un tableau

Exercice 3

Traduire en langage C les exercices précédents.